# Convolutional Neural Networks – II

Structured Notes

CNN vs ANN Efficiency, Inductive

Bias, AlexNet,
Dropout Regularization, and Conceptual Closure

# Contents

**1 CNN vs ANN: Parameter Efficiency** **2**
1.1 Why Fully Connected ANN Struggles with Images . . . 2
1.2 CNN Parameter Structure . . . 2
1.3 Parameter Count Formulas . . . 2
1.4 Why CNN Parameters Depend on Depth but Not on Height/Width . . . 2
1.5 Code: ANN vs CNN Parameter Comparison . . . 3
1.6 Effect of Doubling Image Resolution . . . 4

**2 Spatial Inductive Bias** **4**
2.1 Locality . . . 4
2.2 Translational Invariance . . . 4

**3 Full CNN Architecture Pipeline** **4**
3.1 Tensor Shape Flow . . . 5
3.2 Code: SmallCNN with Shape Tracking . . . 5

**4 Training a CNN** **6**
4.1 Loss Function: Cross-Entropy . . . 6
4.2 Backpropagation in CNN . . . 6
4.3 Code: Single Training Step . . . 6

**5 Dropout Regularization** **7**
5.1 Why Overfitting Happens in CNN . . . 7
5.2 Dropout Mechanism . . . 7
5.3 Dropout Placement in CNN . . . 8
5.4 Code: CNN With and Without Dropout . . . 8

**6 AlexNet: The Breakthrough CNN** **9**
6.1 Before AlexNet (Pre-2012) . . . 9
6.2 The ImageNet Challenge . . . 9
6.3 Key Innovations of AlexNet . . . 9
6.3.1 ReLU Activation . . . 9
6.3.2 Deep Architecture . . . 9
6.3.3 GPU Training . . . 9
6.3.4 Data Augmentation . . . 10
6.3.5 Dropout Regularization . . . 10
6.4 AlexNet Architecture . . . 10
6.5 Code: Simplified AlexNet Feature Extractor . . . 10
6.6 Why AlexNet Was Revolutionary . . . 11

**7 Conceptual Closure: Understanding CNNs** **11**
7.1 The Core Idea . . . 11
7.2 Hierarchical Feature Learning . . . 12
7.3 CNN as Structured Linear Transformations . . . 12
7.4 Why CNNs Generalize Well . . . 12
7.5 Feature Hierarchy Through Layers . . . 12
7.6 Mental Model of CNN . . . 13

**8 Summary** **13**

# 1 CNN vs ANN: Parameter Efficiency

## 1.1 Why Fully Connected ANN Struggles with Images

Consider a $32 \times 32 \times 3$ image. The total number of input values is:

$$32 \times 32 \times 3 = 3072$$

If this flattened input is fed to a fully connected layer with 1000 neurons:

$$\text{Parameters} = 3072 \times 1000 + 1000 = 3{,}073{,}000$$

This produces over **3 million parameters in a single layer**. ANN treats each pixel independently and ignores the spatial structure of the image. Every pixel is connected to every neuron in the next layer.

## 1.2 CNN Parameter Structure

Consider a convolution layer with:

- Input channels $C_{in} = 3$
- Kernel size $K = 3$
- Output channels $C_{out} = 32$

**Conv2D Parameter Count:**

$$\#\text{params} = C_{out}(C_{in} \cdot K^2 + 1) = 32(3 \times 9 + 1) = 32 \times 28 = 896$$

**Parameter Comparison**

| Layer Type | Parameters |
|---|---|
| ANN: one fully connected layer | 3,073,000 |
| CNN: one convolutional layer (32 kernels) | 896 |

This is the power of **weight sharing** and **local connectivity**. Convolution parameter count does **not** depend on image height $H$ or width $W$. It depends only on $C_{in}$, $C_{out}$, and $K$.

## 1.3 Parameter Count Formulas

**For Conv2D:**

$$\#\text{params} = C_{out}(C_{in} \cdot K^2 + 1)$$

**For Fully Connected:**

$$\#\text{params} = d_{out}(d_{in} + 1)$$

Convolution parameter count does **not** depend on image height or width. Fully connected layers scale directly with input dimension.

## 1.4 Why CNN Parameters Depend on Depth but Not on Height/Width

The trainable parameters reside in the **kernel**, not in the image. The kernel dimensions are $C_{in} \times K \times K$. The image dimensions $H \times W$ determine how many times the kernel slides over the input, but do not affect the number of weights.

However, the number of input channels $C_{in}$ (depth) does affect the parameter count because the kernel depth must match the input depth.

**FC Layer at the End**

Though CNN has fewer parameters than ANN in the convolutional layers, the fully connected layer at the end can still dominate the total parameter count. Once feature learning is complete and features are flattened for classification, the parameter count can increase significantly.

Modern architectures address this using **Global Average Pooling (GAP)**, which takes the average of each entire feature map channel, producing one value per channel instead of flattening the full spatial dimensions.

## 1.5 Code: ANN vs CNN Parameter Comparison

**ANN vs CNN Parameter Comparison**

```python
import torch
import torch.nn as nn

class SimpleANN(nn.Module):
    def __init__(self):
        super().__init__()
        self.flatten = nn.Flatten()
        self.fc1 = nn.Linear(3*32*32, 1000)
        self.fc2 = nn.Linear(1000, 10)

    def forward(self, x):
        x = self.flatten(x)
        x = torch.relu(self.fc1(x))
        x = self.fc2(x)
        return x

class SimpleCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        self.fc = nn.Linear(32*16*16, 10)

    def forward(self, x):
        x = torch.relu(self.conv1(x))
        x = self.pool(x)
        x = torch.flatten(x, 1)
        x = self.fc(x)
        return x

def count_params(model):
    return sum(p.numel() for p in model.parameters()
               if p.requires_grad)

ann = SimpleANN()
cnn = SimpleCNN()
print("ANN parameters:", count_params(ann))  # 3,083,010
print("CNN parameters:", count_params(cnn))  # 82,826
```

Listing 1: Comparing parameter counts

### 1.6 Effect of Doubling Image Resolution

If image resolution is doubled ($H \to 2H$, $W \to 2W$):

- **ANN:** Parameters increase by $4\times$ because the flattened input size becomes $4 \times H \times W \times C$.
- **CNN:** No change in convolutional layer parameters, since they depend only on $C_{in}$, $C_{out}$, and $K$.

## 2 Spatial Inductive Bias

**Inductive Bias**

**Inductive bias** refers to the set of assumptions a model makes about the data it processes. CNN encodes specific assumptions about visual data through its architecture, which helps it generalize better with fewer parameters.

CNN assumes two important properties about images:

### 2.1 Locality

Nearby pixels are related. Examples include edges, corners, and textures. Convolution exploits this by using small kernels that process local spatial regions rather than the entire image at once.

### 2.2 Translational Invariance

The same pattern can appear anywhere in the image. For example, a cat ear in the top-left or bottom-right of an image is still a cat ear regardless of its position. CNN achieves this through **weight sharing** — the same kernel is applied across all spatial positions.

**Encoding Inductive Bias**

CNN encodes these assumptions through:

- **Local convolution kernels** — each neuron sees only a small spatial region
- **Weight sharing across space** — the same filter is reused across the entire image

These biases allow CNNs to learn meaningful representations using fewer parameters than fully connected networks.

## 3 Full CNN Architecture Pipeline

A standard CNN classifier follows this pipeline:

$$\text{Conv} \to \text{ReLU} \to \text{Pool} \to \text{Conv} \to \text{ReLU} \to \text{Pool} \to \text{Flatten} \to \text{FC}$$

Each component has a precise role:

| Component | Role |
|---|---|
| Convolution | Extracts local spatial features |
| ReLU | Introduces non-linearity |
| Pooling | Reduces spatial resolution |
| Flatten | Converts feature maps to vectors |
| Fully Connected | Performs classification |

## 3.1 Tensor Shape Flow

Using NCHW format where $x \in \mathbb{R}^{N \times C \times H \times W}$:

**Output spatial dimensions for a convolution layer:**

$$H_{out} = \left\lfloor \frac{H + 2P - K}{S} \right\rfloor + 1 \qquad W_{out} = \left\lfloor \frac{W + 2P - K}{S} \right\rfloor + 1$$

where $P$ = padding, $K$ = kernel size, $S$ = stride.

**Output tensor:** $\mathbb{R}^{N \times C_{out} \times H_{out} \times W_{out}}$

**Pooling** with $2 \times 2$ kernel and stride 2 halves both $H$ and $W$.

## 3.2 Code: SmallCNN with Shape Tracking

**SmallCNN Architecture**

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class SmallCNN(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 16, kernel_size=3,
                               stride=1, padding=1)
        self.pool  = nn.MaxPool2d(kernel_size=2, stride=2)
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3,
                               stride=1, padding=1)
        self.fc1   = nn.Linear(32*8*8, 64)
        self.fc2   = nn.Linear(64, num_classes)

    def forward(self, x):
        # Input: (N, 3, 32, 32)
        x = F.relu(self.conv1(x))  # (N, 16, 32, 32)
        x = self.pool(x)           # (N, 16, 16, 16)
        x = F.relu(self.conv2(x))  # (N, 32, 16, 16)
        x = self.pool(x)           # (N, 32, 8, 8)
        x = torch.flatten(x, 1)    # (N, 2048)
        x = F.relu(self.fc1(x))    # (N, 64)
        logits = self.fc2(x)       # (N, 10)
        return logits
```

Listing 2: CNN classifier with shape annotations

Shape flow for input $(4, 3, 32, 32)$:

| Stage | Shape |
|---|---|
| Input | $(4, 3, 32, 32)$ |
| After conv1 + ReLU | $(4, 16, 32, 32)$ |
| After pool1 | $(4, 16, 16, 16)$ |
| After conv2 + ReLU | $(4, 32, 16, 16)$ |
| After pool2 | $(4, 32, 8, 8)$ |
| After flatten | $(4, 2048)$ |
| After fc1 + ReLU | $(4, 64)$ |
| Logits | $(4, 10)$ |

# 4 Training a CNN

## 4.1 Loss Function: Cross-Entropy

For multi-class classification, the **Cross-Entropy Loss** is used. Given logits $z \in \mathbb{R}^{N \times C}$:

**Softmax** converts logits to probabilities:

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

**Cross-entropy loss:**

$$\mathcal{L} = -\sum_{i=1}^{C} y_i \log(p_i)$$

In PyTorch, `nn.CrossEntropyLoss()` combines LogSoftmax and Negative Log Likelihood internally. Raw logits (not softmax outputs) are passed directly to this loss function.

## 4.2 Backpropagation in CNN

CNN training uses the same principle as ANN:

**Forward pass:**

$$x \to f(x) \to \mathcal{L}$$

**Backward pass:**

$$\frac{\partial \mathcal{L}}{\partial W}$$

Convolution is a linear operation, so gradients flow through CNN exactly as in ANN, but respecting **weight sharing**. Weight sharing means each kernel update accumulates gradients from all spatial positions where the kernel was applied.

## 4.3 Code: Single Training Step

**CNN Training Step**

```
import torch.optim as optim

model = SmallCNN(num_classes=10)
```

```
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

inputs = torch.randn(8, 3, 32, 32)
targets = torch.randint(0, 10, (8,))

model.train()
outputs = model(inputs)           # forward pass
loss = criterion(outputs, targets)
optimizer.zero_grad()
loss.backward()                   # backward pass

# Inspect gradient magnitudes
for name, param in model.named_parameters():
    if param.grad is not None:
        print(f"{name:15s} | grad mean: "
              f"{param.grad.abs().mean():.6f}")

optimizer.step()                  # update weights
```

Listing 3: Training loop with gradient inspection

**CNN Optimization**

CNN optimization is not fundamentally different from ANN. The difference is in the **parameter structure** (local connectivity and weight sharing), not in the learning rule. Gradient magnitudes may differ across layers — convolution gradients accumulate over spatial locations.

# 5 Dropout Regularization

## 5.1 Why Overfitting Happens in CNN

Even with fewer parameters than a fully connected ANN, CNNs can still overfit because:

- The model can memorize patterns specific to the training set
- Fully connected layers near the end can become memorization-heavy
- Small datasets or noisy labels cause features to become overly specialized

**Overfitting Signature**

- Training loss keeps decreasing
- Validation loss stops improving or increases

## 5.2 Dropout Mechanism

**Dropout**

Dropout randomly "turns off" a fraction of activations during training. If dropout probability is $p$:

- With probability $p$, a neuron output becomes 0
- With probability $1 - p$, it stays active

During training, a binary mask is sampled:

$$m \sim \text{Bernoulli}(1 - p)$$

and applied element-wise:

$$\tilde{h} = m \odot h$$

This forces the network to avoid relying too heavily on any single neuron, encouraging **redundancy** and more **robust representations**.

## 5.3 Dropout Placement in CNN

- **Strongest impact:** Before or between fully connected layers
- **Optional:** After convolution blocks (less common in small CNNs)
- **Beginner rule:** Apply dropout after flatten or after the first FC layer

**Train vs Eval Mode**

- Dropout is **ON** during training (`model.train()`)
- Dropout is **OFF** during evaluation/testing (`model.eval()`)

In training mode, outputs are stochastic (different each forward pass). In eval mode, outputs are deterministic (identical each forward pass).

## 5.4 Code: CNN With and Without Dropout

**Dropout Comparison**

```python
class CNN_WithDropout(nn.Module):
    def __init__(self, num_classes=10, p=0.5):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 16, 3, padding=1)
        self.pool  = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(16, 32, 3, padding=1)
        self.fc1   = nn.Linear(32*8*8, 64)
        self.drop  = nn.Dropout(p=p)
        self.fc2   = nn.Linear(64, num_classes)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = torch.flatten(x, 1)
        x = F.relu(self.fc1(x))
        x = self.drop(x)    # dropout after FC1
        x = self.fc2(x)
        return x

model = CNN_WithDropout(p=0.5)

# Train mode: outputs differ each pass
model.train()
out1 = model(x)
out2 = model(x)
print(torch.allclose(out1, out2))  # False

# Eval mode: outputs are identical
model.eval()
out1 = model(x)
out2 = model(x)
print(torch.allclose(out1, out2))  # True
```

Listing 4: CNN with dropout after FC1

# 6 AlexNet: The Breakthrough CNN

## 6.1 Before AlexNet (Pre-2012)

Before 2012, image recognition relied mostly on:

- **Handcrafted features** — features designed manually by domain experts
- **Classical ML algorithms** — applied on top of extracted features

The typical pipeline was: **Image → Manual Feature Extraction → Classifier**

Examples of handcrafted features include SIFT, HOG, and SURF. These methods required **manual feature engineering**.

Deep learning existed at the time, but training deep networks was difficult because:

- Networks were hard to optimize
- Activation functions (sigmoid, tanh) saturated
- GPU training was uncommon (only CPU training was available)
- Large labeled datasets were limited

## 6.2 The ImageNet Challenge

**ImageNet**

ImageNet is a large-scale image classification dataset:

- Approximately **1.2 million** training images
- **1,000** object classes
- Goal: given an image, predict the correct class
- Evaluation metric: **Top-5 error rate**

| Model | Top-5 Error Rate |
|---|---|
| Best pre-AlexNet (2011) | $\approx 26\%$ |
| AlexNet (2012) | $\approx 15\%$ |
| EfficientNet (later) | $\approx 7\%$ |

This massive improvement shocked the computer vision community.

## 6.3 Key Innovations of AlexNet

AlexNet succeeded because of several engineering innovations:

### 6.3.1 ReLU Activation

AlexNet was the first architecture to use ReLU activation instead of sigmoid or tanh:

$$\mathrm{ReLU}(x) = \max(0, x)$$

Benefits: faster training and reduced vanishing gradient problem. ReLU avoids saturation and allows stronger gradient propagation compared to sigmoid and tanh.

### 6.3.2 Deep Architecture

AlexNet used **8 learnable layers**: 5 convolution layers and 3 fully connected layers. Only convolution and fully connected layers contain learnable parameters. This depth allowed **hierarchical feature learning**.

### 6.3.3 GPU Training

AlexNet was trained on **two GPUs in parallel**, which dramatically reduced training time. Training a large CNN on millions of images is computationally very expensive and would have taken months on CPUs alone.

### 6.3.4 Data Augmentation

Training images were augmented with:

- Random crops
- Horizontal flips
- Color variations (brightness changes)

From a single image, different orientations and versions are created to increase the effective dataset size and improve generalization.

### 6.3.5 Dropout Regularization

Dropout was applied in the fully connected layers to reduce overfitting.

## 6.4 AlexNet Architecture

**Input:** $227 \times 227 \times 3$

**Layer structure:**

Conv1 → ReLU → MaxPool

Conv2 → ReLU → MaxPool

Conv3 → ReLU

Conv4 → ReLU

Conv5 → ReLU → MaxPool

Flatten

FC1 → ReLU → Dropout

FC2 → ReLU → Dropout

FC3 → Softmax

**Pooling Pattern in AlexNet**

Pooling occurs after Conv1, Conv2, and Conv5 — not after Conv3 and Conv4. After layers without pooling, the spatial dimensions remain unchanged. MaxPool with kernel 3 and stride 2 is used (not the standard $2 \times 2$ with stride 2).

The convolution layers progressively extract increasingly abstract features: **Edges → Textures → Object Parts → Object Concepts**.

## 6.5 Code: Simplified AlexNet Feature Extractor

**Mini AlexNet**

```
class MiniAlexNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=11,
                      stride=4, padding=2),
            nn.ReLU(),
            nn.MaxPool2d(3, 2),

            nn.Conv2d(64, 192, 5, padding=2),
            nn.ReLU(),
            nn.MaxPool2d(3, 2),
```

```
13
14              nn.Conv2d(192, 384, 3, padding=1),
15              nn.ReLU(),
16
17              nn.Conv2d(384, 256, 3, padding=1),
18              nn.ReLU(),
19
20              nn.Conv2d(256, 256, 3, padding=1),
21              nn.ReLU(),
22              nn.MaxPool2d(3, 2)
23          )
24
25      def forward(self, x):
26          return self.features(x)
27
28  model = MiniAlexNet()
29  x = torch.randn(1, 3, 227, 227)
30  out = model(x)  # feature maps from last layer
```

Listing 5: Simplified AlexNet-like architecture

### 6.6 Why AlexNet Was Revolutionary

AlexNet demonstrated that **deep CNN + large dataset + GPU training** can outperform traditional computer vision methods. This triggered the modern deep learning revolution.

After AlexNet, the following architectures emerged:

- **VGGNet** (2014)
- **GoogLeNet / InceptionNet** (2014)
- **ResNet** (2015)

CNN became the dominant architecture for vision tasks.

**Historical Context**

AlexNet did not invent CNNs. CNNs existed since the 1990s — **LeNet** was the first CNN architecture. However, AlexNet proved that CNNs scale to real-world datasets when combined with:

1. Large data (ImageNet)
2. GPU compute
3. Better activation functions (ReLU)
4. Regularization techniques (Dropout)

AlexNet was the tipping point in the era of deep learning.

## 7 Conceptual Closure: Understanding CNNs

### 7.1 The Core Idea

A Convolutional Neural Network is fundamentally a sequence of **structured linear operations plus non-linearities**:

$$x \to \text{Conv} \to \text{ReLU} \to \text{Pool} \to \text{Conv} \to \cdots \to \text{FC}$$

Each convolution layer learns filters that detect spatial patterns such as edges, corners, textures, and object parts. These learned filters allow the network to automatically extract features from raw images.

## 7.2 Hierarchical Feature Learning

CNNs learn representations hierarchically:

| Layer Depth | Features Learned |
|---|---|
| Early layers | Simple patterns: edges, gradients, color contrasts |
| Middle layers | Combinations of edges: textures, shapes, corners |
| Deeper layers | Semantic concepts: eyes, wheels, faces |
| Final layers | Object-level features for classification |

Deeper layers combine simple features to learn complex shapes and objects. This hierarchy allows CNNs to move from **low-level signals to high-level concepts**.

## 7.3 CNN as Structured Linear Transformations

Recall ANN: $y = Wx + b$. CNN performs a similar linear transformation, but with two structural constraints:

- **Local connectivity:** Each neuron sees only a small spatial region
- **Weight sharing:** The same filter is reused across the entire image

This reduces parameter count and improves generalization. CNN layers are essentially **structured versions of linear layers**.

## 7.4 Why CNNs Generalize Well

CNNs generalize better than fully connected networks on images because they encode important assumptions (inductive biases):

- **Locality:** Nearby pixels are related
- **Translational invariance:** The same object can appear anywhere in the image
- **Parameter sharing:** The same feature detector is reused across the image

## 7.5 Feature Hierarchy Through Layers

As data flows through a CNN:

- Number of **channels increases** layer by layer (to represent more feature types)
- **Spatial resolution decreases** due to pooling or strided convolution (reducing computation and capturing larger context)

**Feature Hierarchy Code**

```python
class HierarchyCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 16, 3, padding=1)
        self.conv2 = nn.Conv2d(16, 32, 3, padding=1)
        self.conv3 = nn.Conv2d(32, 64, 3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)

    def forward(self, x):
        x = self.pool(torch.relu(self.conv1(x)))
        # (1, 16, 16, 16)
        x = self.pool(torch.relu(self.conv2(x)))
        # (1, 32, 8, 8)
        x = torch.relu(self.conv3(x))
        # (1, 64, 8, 8)
```

```
16          return x
```

Listing 6: Observing spatial reduction and channel increase

### 7.6 Mental Model of CNN

A CNN operates as a feature extraction pipeline:

**Image → Edge Detectors → Texture Detectors → Part Detectors → Object Detectors → Classifier**

This pipeline is the reason CNNs work well for vision tasks.

**Parameter Explosion Risk**

If CNN used fully connected layers from the start instead of convolutions, the result would be **parameter explosion** — an impractically large number of weights that would overfit quickly and be computationally infeasible to train.

## 8 Summary

- **CNN vs ANN efficiency:** CNN uses weight sharing and local connectivity to dramatically reduce parameter count compared to fully connected ANN on images. A single conv layer with 32 kernels has 896 parameters vs. over 3 million for an equivalent FC layer.
- **Inductive bias:** CNN encodes locality and translational invariance as assumptions, enabling better generalization with fewer parameters.
- **CNN pipeline:** Conv → ReLU → Pool → Flatten → FC. Output spatial dimensions follow the formula $H_{out} = \lfloor (H + 2P - K)/S \rfloor + 1$.
- **Training:** Uses cross-entropy loss with raw logits. Backpropagation works identically to ANN but respects weight sharing.
- **Dropout:** Randomly zeroes activations during training using a Bernoulli mask ($\tilde{h} = m \odot h$). Applied between FC layers. Must be turned off during evaluation.
- **AlexNet (2012):** First deep CNN to win ImageNet (15% vs 26% error). Innovations: ReLU, 8-layer depth, GPU training, data augmentation, dropout. Triggered the deep learning revolution.
- **Historical context:** LeNet was the first CNN (1990s). AlexNet proved CNNs scale with large data + GPU + better activations + regularization.
- **Hierarchical learning:** Early layers detect edges, middle layers textures, deep layers semantic objects. Channels increase and spatial resolution decreases through the network.
- **Modern improvement:** Global Average Pooling (GAP) replaces large FC layers to reduce parameter count at the classification stage.